

Ogham Manual

Full user manual for Ogham — a dual-voice bytebeat synthesizer for Eurorack.

UPDATED 10 AUGUST 2026

A dual-voice bytebeat synthesizer for Eurorack, built on the Electro-Smith Daisy Seed.

1. Introduction

Ogham is a Eurorack voice that turns tiny mathematical formulas into music. Instead of oscillators and wavetables, it runs bytebeat expressions — single lines of integer arithmetic that, evaluated over time, produce surprisingly rich rhythmic and melodic patterns. Ogham hosts a bank of these formulas, lets you play two of them at once, shape them with a bipolar Tone macro and a 3-stage modulation FX chain, and clock, pitch, and modulate them from the rest of your system.

At a glance

- 100 function slots (**F0** – **F99**), each with two performable parameters (A & B). All hundred are filled, grouped by character — twenty each of textural, noise, percussive, rhythmic and melodic (see Section 4). A special **AA** slot holds a built-in A=440 Hz tuning reference.
 - Two independent voices (Out 1 / Out 2), each playing its own function.
 - Out 2 “drone/freeze” — decouple Out 2 into an independent, self-sustaining voice that ignores the knobs, clock, pitch, and FX.
 - Bipolar Tone macro — one knob from clean through low-pass filtering, wavefolding, saturation, sample-rate reduction, and resonant band-pass grit.
 - 3-stage modulation FX — Chorus → Flanger → Phaser, each with a clean and a characterful variant, in series or parallel.
 - Mode switch: Clk ↔ VOct on a shared jack.
 - Clk — external tempo/clock sets the bytebeat playback rate.
 - VOct — 1 V/oct pitch: the engine hard-syncs to play clean tuned notes.
 - Hard-sync Sync input — sample-accurate waveform reset, usable up to audio rates.
 - CV over A and B (± 5 V, summed with the knobs); either CV can be re-routed to the Tone macro instead of its parameter.
 - Internal LPG — a low-pass gate on the output, plucked from the Sync input: sharp attack, decay from 2 ms to 20 s, filter and VCA moving together.
 - Selectable ENV Out — an envelope follower of either voice, *or* a raw bytebeat used as a slow DC modulation/LFO source; plus an EOC pulse out.
 - Voice selections, the full effects/settings menu, and all options persist across power cycles; a one-gesture factory reset.
-

2. The panel

Ogham packs a lot into a small panel: five knobs (one is the Func encoder), a mode switch, a 4-digit display, four inputs, and four outputs. The map below names every control; the detailed behaviour of each is in Section 5, *Controls — full reference* (the numbers here match the references there).



Control legend

#	Control	Type	In short
1	Func	Rotary encoder (with push)	Select each voice's function; long-press opens the Menu; short-click switches voice
2	A	Knob (+ CV)	Formula Parameter A (0–255), shared by both voices
3	B	Knob (+ CV)	Formula Parameter B (0–255), shared by both voices
4	Rate / Fine	Knob	Playback rate $1/64\times$ – $64\times$ (a quantized clock $\times/\div$ when a clock is patched); ± 1 -octave fine tune in VOct mode

#	Control	Type	In short
5	Tone	Knob	Bipolar tone macro — clean at 12 o'clock, filtering/fold CCW, crush/resonance CW
6	Mode	Toggle switch	Selects the shared jack (11): Clk (tempo) or VOct (pitch)
7	Display	4-digit 7-segment	Current function, menu fields, parameter flashes, clean-center dot
8	CV A	Input (± 5 V)	Modulates Parameter A (summed with knob 2); can be routed to Tone
9	CV B	Input (± 5 V)	Modulates Parameter B (summed with knob 3); can be routed to Tone
10	Sync	Input (trigger/gate)	Hard-sync reset — restarts the waveform (audio-rate capable)
11	Clk/VOct	Input (shared)	Tempo clock or 1 V/oct pitch, per the Mode switch (6)
12	Out 1	Audio output	Voice 1 (primary) — drives the EOC / default envelope
13	Out 2	Audio output	Voice 2 — independent function; can be frozen as a drone
14	ENV Out	CV output (DC-coupled)	Envelope follower of a voice, or a raw bytebeat DC/LFO — mode set in the Menu
15	EOC	Gate output	End-of-cycle pulse (~10 ms), BPM-synced from Voice 1

Reading the display (7): functions show as **1-NN** / **2-NN** (voice + function, 0-based), **1-AA** for the tuning reference, and various codes in the Menu (Section 5). The far-right decimal point lights whenever the Tone knob (5) is at its clean center.

3. What is bytebeat?

Bytebeat is a style of music discovered and popularised by Viznut (Ville-Matias Heikkilä) in 2011. The whole instrument is a single expression:

```
output = f(t)          // f returns an 8-bit value 0-255
```

`t` is a counter that increments at a fixed sample rate (Ogham uses 8 kHz for its authored formulas). You feed `t` through arithmetic and bitwise operators — shifts, AND/OR/XOR, multiply, divide, modulo — and the low 8 bits of the result are sent to the DAC as audio. No envelopes, no note lists; the *structure of the number sequence itself* becomes rhythm, melody, and timbre.

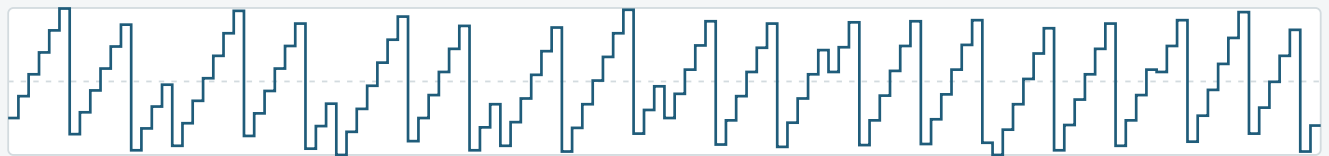
A classic example is `t*(((t>>12)|(t>>8))&(63&(t>>4)))` — a handful of characters that produce an evolving arpeggio. It is worth seeing what that actually looks like, because it explains why bytebeat behaves the way it does:

One expression, three time scales

`t * (((t>>12) | (t>>8)) & (63 & (t>>4)))` — the classic bytebeat one-liner, at 8 kHz

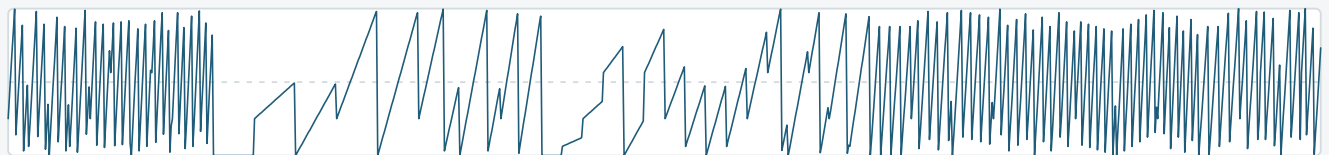
128 steps — 16 ms

the individual numbers, stepping



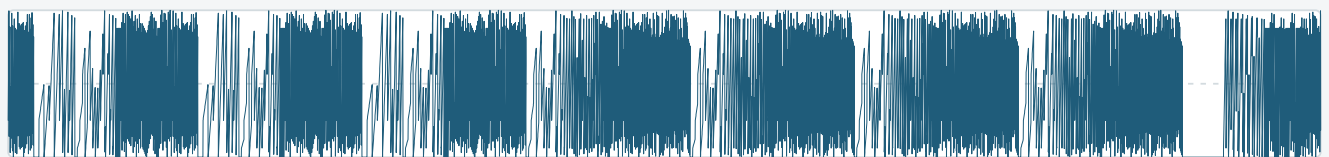
1,024 steps — 0.13 s

a repeating wave, so a pitch



8,192 steps — 1 s

the pattern moving through it



Same expression in all three — no oscillator, no sequencer, just `t` counting up

Zoom right in and you see the individual numbers stepping — that is the waveform, and so the timbre. Pull back and those steps form a repeating wave, which is the pitch. Pull back further and the wave itself is being reshaped, and that is the rhythm and the melody.

All three come from the same expression. There is no oscillator and no sequencer to adjust separately, which is what makes bytebeat so compact — and why small changes to the constants produce wildly different results, and why it is both fascinating and hard to control.

How Ogham plays it

- Each formula runs at its base rate (8 kHz for the authored bank), then is linearly interpolated up to the 48 kHz codec output for a smoother, less aliased sound.
 - A 32.32 fixed-point phase accumulator bridges the two rates and provides the variable playback rate (the Rate / Fine knob, or an external clock).
 - All maths use JavaScript `ToInt32` semantics (shifts masked to `& 31`, result masked to `& 0xFF`) so formulas behave exactly as they do in the browser bytebeat tools they were designed in.
-

4. The functions — 100 slots

Ogham addresses 100 function slots, shown on the display as **F0** – **F99** (the numbering is 0-based: the first function reads **1-00**). Turning the Func encoder (1) moves through them; the selector does not wrap (see Section 5).

What's in the slots today

Slots	Contents
F0 – F19	Textural (8 kHz base rate) — drones, sweeps and evolving beds.
F20 – F39	Noise — grit, hiss and broken-radio textures.
F40 – F59	Percussive — hits, clicks and drum-like transients.
F60 – F79	Rhythmic — patterns and grooves that repeat.
F80 – F99	Melodic — pitched lines and harmonic movement.
AA	The A=440 Hz tuning reference — a special slot past F99 (see Section 5, <i>A440 reference</i>).

Appendix A at the end of this manual lists the bank as shipped with your firmware — each function's name and number, its actual bytebeat expression, and what the A and B knobs change in it. The bank is revised between firmware versions, so the appendix is the authority; the rest of this manual describes how the functions behave rather than naming individual ones.

Discovery

The raw formulas came out of a purpose-built bytebeat research toolchain:

- GPU search — evaluating huge numbers of candidate expressions in parallel.
- Neural fitness — a model trained on human preference data to score how “musical” a candidate sounds, steering the search toward usable results.

This surfaces expressions with interesting rhythmic and harmonic behaviour from an otherwise enormous and mostly-noisy search space.

Parameterisation (choosing A & B) — evolved in

To make a formula *performable*, two of its numbers have to become the live parameters A and B. Taking a finished formula and promoting two of its constants turns out to be the hard part:

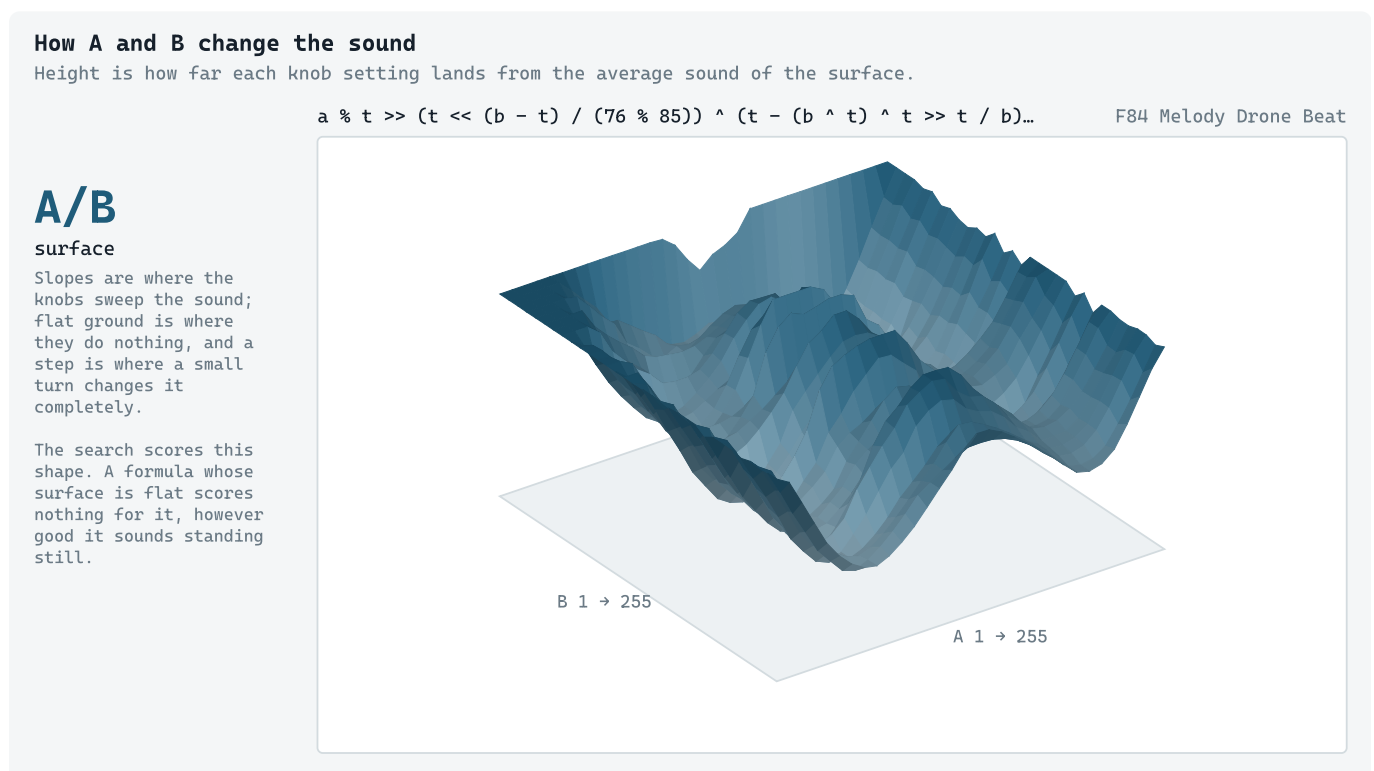
Most constants in a bytebeat formula are load-bearing — change one by 1 and you get silence, noise, or an unrelated sound. Only a few constants vary *musically*.

We tried automating that (sweeping every constant on the GPU and scoring for smoothness + variance). It mostly failed — of 75 formulas tested, 66 scored zero. The reasons are fundamental to bytebeat:

1. Integer discontinuity — changing a shift from `>>8` to `>>9` halves the pitch; there's no smooth in-between, so a naive search sees only abrupt jumps.
2. Co-evolved constants — in a discovered formula every constant is tuned to work *together*; replace one and the interference patterns collapse.

Both problems come from the same place: the formula was finished before anyone asked whether it could be played. So the search stopped producing formulas and then parameterising them, and started **evolving a and b as part of the expression**. They are variables the search can use anywhere, from the first generation — not constants promoted afterwards.

That changes what a candidate has to be good at. Fitness no longer scores one sound; it renders the formula across the whole A/B plane and scores the *set* of sounds that come back — how far apart they are, how much of the plane is dead, and whether neighbouring settings are related or unrelated. A formula that sounds wonderful at one setting and identical everywhere else now scores badly, because that is a knob you cannot play.



The surface above is one function's response, measured the way the search measures it: every point is a different A and B, and the height is how far that setting lands from the average sound of the whole plane. Slopes are playable territory. Flat ground is a knob doing nothing. The steps are the integer discontinuity from the first problem above — it does not go away, it is simply now something a candidate is scored on rather than something discovered afterwards.

The co-evolved-constants problem does not go away either; it stops mattering, because nothing gets replaced. **a** and **b** were load-bearing from the start, and the rest of the expression grew around them.

The result: on Ogham, A and B always do something for each formula — but *what* they do is formula-specific, and part of the instrument is exploring it. Expect plateaus and sudden jumps as well as smooth sweeps; all three are visible in the surface above, and all three are in the bank. (The Smooth  option, Section 5, tames the abrupt jumps when you want glide.)

Parameter behaviour on the module

- A and B are each 0–255, set by the knobs (and CV), and fed to both voices.
 - There are no hidden per-formula ranges or stored defaults — the knob position *is* the value, always. Selecting a new function keeps your current A/B (it just reinterprets them), which maximises exploration and keeps the two voices' parameters predictable.
 - Because formulas use A/B differently, expect some knob positions to be silent or harsh on some formulas — that's the territory; sweep to find the sweet spots.
 - A few formulas mention only one of A and B, or neither; on those slots the other knob does nothing. Appendix A shows which, since the expressions are printed exactly as the module evaluates them.
-

5. Controls — full reference

This section details every control. Numbers in bold refer to the panel map in Section 2.

Func encoder (1) — the heart of the interface

The Func encoder has two modes. SELECT mode chooses each voice's function; MENU mode is the effects & settings menu (see below). A long press toggles between them.

Gesture	In SELECT mode	In MENU mode
Turn	Set the selected voice's function (F0 – F99 , then AA ; no wrap)	Move through the menu fields (no wrap, accelerating)
Short click	Switch voice (Out 1 ↔ Out 2)	Edit / commit the current field
Long press (~0.6 s)	Enter the Menu	Exit the Menu (from anywhere, incl. mid-edit)
Hold during power-on	Factory reset (functions → F0 , menu → defaults)	

- **No wrap-around:** the selector clamps at the ends — a hard crank reliably slams to **F0** at one end and **AA** at the other, instead of looping around. The Menu's field list clamps the same way.
- **Acceleration:** turn slowly for single steps (one function per detent); spin faster to jump several at a time (roughly $\times 2$ / $\times 4$ / $\times 8$ with turn speed), so you can cross the 100-slot range quickly and still land precisely. The Menu's fields accelerate too, on a gentler curve (up to $\times 3$) suited to its much shorter list — a brisk crank crosses the whole Menu in a few detents, while a deliberate turn always moves exactly one field.
- **Editing menu values:** in MENU mode, turn to move through fields; click to start editing the highlighted field (its value flashes while editing); turn to change it; click again to commit.
- **Display:** **1-07** = Out 1 playing function 7 (0-based; first slot is **1-00**); **2-14** = Out 2 playing function 14; **1-AA** = Out 1 on the A440 reference.

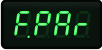
The Menu (1) — effects chain + settings

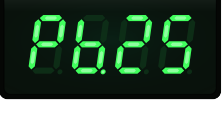
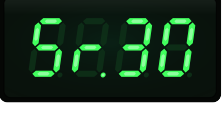
A long press on Func opens the Menu. It holds the FX chain (the modulation effects) plus a set of global settings (the ENV-output mode with its slew and hold, the internal LPG, CV→Tone routing, parameter smoothing and the Out 2 drone).

Navigate with turn; click to edit a field (its value flashes while editing); turn to change it; click to commit; long-press to exit. The field list does not wrap — it stops at the first and last field — and accelerates with turn speed, so a crank reliably slams to either end. Everything in the Menu is saved to flash, and the performance knobs (A, B, Rate / Fine, Tone) keep working while it's open.

The Menu also remembers where you were: leave it and come back and you land on the same field, so nudging one parameter between takes doesn't mean re-navigating from the top each time. It always reopens ready to navigate, never mid-edit. This is deliberately not saved to flash — a power cycle puts you back at the first field.

The FX chain is three modulation stages — Chorus → Flanger → Phaser — inserted after the Tone macro (Out 1 / Out 2 are a stereo pair, each with its own effect state). FX-stage fields read as **T x . NN** — effect letter (C/F/P) · sub-field (L/t/a/b) · value.

Display	Field & what it does
	Global on/off — enable or bypass the whole FX chain:  on,  off.
	Chain topology —  series (each stage feeds the next), or  parallel (all three process the dry signal; wet outputs summed).
Chorus stage (C)	
	Level — wet/dry mix, 0–99 (0 = bypassed).
	Type —  clean doubler, or  Ensemble (3 detuned voices, lush & wide).
	Param a — rate.
	Param b — depth.
Flanger stage (F)	
	Level — wet/dry mix, 0–99 (0 = bypassed).
	Type —  gentle sweep, or  Barber-pole (endless rising/falling sweep).
	Param a — speed + direction (in Barber-pole, 50 = stopped).

	Param b — feedback.
Phaser stage (P)	
	Level — wet/dry mix, 0–99 (0 = bypassed).
	Type —  4-pole sweep, or  Bi-phase (8-pole, two interacting LFOs).
	Param a — rate.
	Param b — 2nd-LFO ratio.
ENV Out & global settings	
	ENV Out mode (jack 14) —  /  envelope follower of Out 1/Out 2, or  /  the raw Out 1/Out 2 bytebeat as DC (a bytebeat LFO).
	ENV Out slew, rise — 0–99, off to about 5 s , smoothing the output as it moves <i>up</i> . It means the same number of milliseconds in every mode, so once you've found a setting it stays put as you change Rate.
	ENV Out slew, fall — the same, applied as the output moves <i>down</i> . Separate from rise, so a quick attack can sit alongside a long release (or the reverse). In the DC modes Ogham also restores the level automatically: heavy smoothing would otherwise shrink the voltage toward its average and go quiet.
	ENV Out hold — sample-and-hold on the DC modes only:  off (follows every step), or  a window of 2–256 steps, so the voltage updates once per window and holds in between. The steps stay in time with the pattern. Runs before Slew, so the two together glide from one held value to the next.
	LPG — the internal low-pass gate, plucked by the Sync input. One field for both on/off and decay:  off (the output runs open), or  1–99 to switch it on with that decay, from 2 ms (a tick) to 20 s (a long swell). The curve has a knee at 50 , which is 10 s : the lower half covers 2 ms – 10 s, and the whole upper half is given over to 10 – 20 s in very fine steps. 20 ≈ 60 ms, 30 ≈ 330 ms, 40 ≈ 1.8 s. See <i>The LPG</i> below.
	Tone CV — route CV to the Tone macro instead of a parameter:  normal,  CV A sweeps Tone (Param A = knob only),  CV B sweeps Tone (Param B = knob only).

	Smooth (q) — A/B parameter interpolation: 9oFF off (raw), or a grid step 90 16 4–128 that crossfades the formula output between grid points so A/B sweeps glide (smaller = finer).
	Drone — Out 2 decouple / freeze: don freeze Out 2 as an independent drone (own pitch + A/B, bypasses Tone + FX), doFF live. The frozen pitch + A/B persist across power cycles.

A & B knobs (2, 3) — formula parameters (knob + CV)

- Range: 0–255 each, shared by both voices.
- Knob + CV are summed in hardware. The knob sets the base value; CV offsets it bipolarly around that point; the sum is clamped 0–255.
- CV is ± 5 V bipolar (8, 9). With the knob centered, ± 5 V sweeps the full range (with a little headroom). With the knob toward an extreme, CV has room only in the *other* direction (e.g. knob full up → only negative CV does anything).
- Moving A or B briefly flashes its value on the display (**A nnn** / **b nnn** , ~1 s), showing the combined knob + CV value. The flash is triggered by turning the knob (not by CV alone), but once shown it tracks live CV changes until it times out.
- Unpatched CV jack = 0 V, so the knob alone sets the value.
- Either channel's CV can be re-routed to the Tone macro — see *Tone CV*.

Rate / Fine knob (4) — playback rate, or V/oct fine-tune

The label reflects its two jobs, one per Mode:

- **In Clk/normal mode — Rate:** playback rate, $1/64\times$ – $64\times$ exponential, 12 o'clock = $1\times$ (native speed). It stretches/compresses time for both voices (it scales the shared phase accumulator) — a coarse pitch/tempo control. With an external clock present, the knob changes character: it becomes a quantized clock multiplier/divider (see *Clk input*).
- **In VOct mode — Fine:** the knob is a bipolar ± 1 -octave fine tune (12 o'clock = 0). Pitch then comes from the V/oct CV; the knob offsets it up to an octave either way.

Tone knob (5) — bipolar tone macro

One knob, two opposite characters, clean at 12 o'clock — the only position with no processing, confirmed by the far-right dot on the display (7). It applies equally to both voices (unless Out 2 is decoupled). Output level is fixed at unity — use a downstream VCA/mixer. Both endpoints and the center are calibrated to the physical knob positions.

The display (7) doubles as a clean-center indicator. While the knob sits in the center detent — the only position with no processing — the far-right decimal point lights on the normal function display: (Tone clean). Move it off center in either direction and the dot goes dark: (Tone active). It's a reliable at-a-glance cue that Tone is truly bypassed — easier to trust than eyeballing the knob against the panel.

From center...	Early travel	Further travel
CCW ↺	Low-pass filter sweeps closed (smooths/darkens)	Drive → wavefolder → saturation (re-synthesises rich harmonics from the smoothed wave)
CW ↻	Sample-rate reduction + saturation ramp in, and a resonant band-pass opens (cutoff rising, bandwidth narrowing)	Adds overdrive/distortion grit; the band-pass sweeps up into a tight resonant peak (gritty, vocal, crushed)

The CW side stacks four stages — sample-rate reduction (exponential), soft saturation, overdrive/distortion (drive into a hard clip), and a resonant band-pass sweep whose cutoff and resonance rise across the throw. So CCW = *warm/folded*, CW = *gritty resonant crush*, center = *clean*. Via Tone CV, CV A or B can sweep this macro hands-free.

Mode switch (6) — Clk ↔ VOct

A toggle that selects what the shared Clk/VOct jack (11) does:

- Clk — the jack is an external tempo/clock input (see *Clk input*).
- VOct — the jack is a 1 V/oct pitch input (see *VOct input*).

The switch changes the jack's role immediately; the rest of the panel keeps working as normal.

Sync input (10) — hard-sync reset

- A rising edge resets **t** to 0 for both voices (restarts the waveform).
- Implemented as a hardware interrupt, applied sample-accurately — so it works cleanly from slow triggers all the way up to audio-rate hard sync: drive it from an oscillator and Ogham locks to that pitch with bytebeat timbre.
- Catches narrow trigger pulses (edge-triggered, not width-dependent).
- Exception: a decoupled Out 2 (drone) free-runs and ignores Sync; the reset still applies to Out 1.
- The same edge also plucks the internal LPG, when it is switched on (below).

The LPG — internal low-pass gate

Ogham has a low-pass gate on its output, off by default and switched on from the Menu (**Lp.NN**). It is the classic Buchla-style pairing: one envelope drives a VCA *and* a low-pass filter together, so a quiet moment is also a dark one. That coupling is what makes it sound like a struck object decaying rather than a volume fade — Ogham's raw bytebeat, bright and relentless by nature, becomes a plucked, resonant body.

The filter deliberately runs *ahead* of the VCA: it darkens faster than the note gets quieter, so brightness collapses early in the decay while the level is still up. That is what gives the gate its bite rather than a polite fade.

- **Trigger:** the Sync input. Every rising edge fires the gate — the same edge that resets the waveform, so a single trigger both restarts the timbre and plucks it. Patch a trigger sequencer into Sync and Ogham plays notes.
- **Attack:** fixed and sharp (under a millisecond), so the transient always has bite. It shortens further at the fastest decays, so a 2 ms setting really is a tick.
- Decay shares the **Lp.NN** Menu field with on/off — **Lp.oF** is off, 1–99 switches it on with that decay, 2 ms to 20 s. The curve is deliberately lopsided: a knee at 50 = 10 s puts all the percussive material in the lower half (0 ≈ 2 ms, 20 ≈ 60 ms, 30 ≈ 330 ms, 40 ≈ 1.8 s) and hands the entire upper half to long decays. The figure is the real length of the note — the envelope reaches actual silence at that time rather than trailing off. The fall is also flatter than a textbook exponential, so a note holds its body and then drops away instead of collapsing the instant it is struck.
- Retriggering mid-decay restarts the attack from the current level, so fast trigger streams swell rather than click.

Two things worth knowing:

- With the LPG on and *nothing* patched to Sync, the module is silent — the gate is shut and waiting. That is the nature of an LPG, not a fault. Switching it on in the Menu plucks it once so you can hear the setting; after that it needs triggers.
- A decoupled Out 2 (drone) is not gated. The drone deliberately ignores Sync, so it keeps running underneath while Out 1 is plucked — one patch gives you a sustained drone and a rhythmic voice.

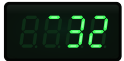
The LPG sits last in the chain, after Tone and the FX, so it gates the finished sound. ENV Out follows the gated audio, which makes it a ready-made envelope for the rest of your rack.

Clk input (11, Mode = Clk) — tempo / clock

A clock input via a comparator (accepts any Eurorack clock/gate/audio level). It sets the bytebeat playback rate: the incoming clock rate plays at 1× (native speed) and a faster clock plays faster. Edges are timed in hardware (stable, jitter-free) and median-filtered.

Quantized multiply / divide (Rate / Fine knob): with a clock present, the Rate / Fine knob (4) stops being a continuous trim and instead snaps to clean power-of-two ratios of the clock rate. 12 o'clock = 1× (play at the clock rate); clockwise multiplies (×2, ×4, ×8, ×16, ×32); counter-clockwise divides (÷2 ... ÷32). Because "rate" time-stretches the whole formula, a multiply/divide shifts pitch and timbre together with the tempo — it is not a pure tempo divider.

As you turn the knob under a clock, the display briefly flashes the selected ratio: a multiply is a bare number

—  (×4) — and a divide is the same number with a small bar to its left —  (÷8),  (÷32).

Freeze / hold on cable-pull: when a running clock stops — you pull the cable — Ogham holds the last tempo and keeps playing at it, rather than jumping to whatever the knob says. This is the least-surprising result when a cable is yanked mid-pattern.

- **Re-lock:** if an edge arrives later (a slow clock, or a resumed sequencer), Ogham re-syncs to it and tracks again — so a genuinely slow clock keeps working; only true silence stays frozen.
- **Take back control:** move the Rate / Fine knob past a small deadband to drop the held clock and return to continuous rate control.
- **Consequence:** if you pause your sequencer (the clock genuinely stops), Ogham keeps playing at the held tempo instead of stopping — nudge the knob to go manual.
- Because the held tempo simply continues, there is no audible “revert” glitch on a pull, and slow clocks no longer risk a dropout between their beats. (The jacks still have no cable-detect switch — a future hardware revision adds true detection — but freezing sidesteps the old slow-clock timeout entirely.)

VOct input (11, Mode = VOct) — pitch

With the Mode switch on VOct, the shared jack becomes a 1 V/oct pitch CV. The engine internally hard-syncs the waveform to the pitch, so the output is a clean, tuned tone that tracks 1 V/oct over roughly 5 octaves.

- 0 V = C1 (the base note), and the input is unipolar 0–5 V (negative voltages are floored at C1). 0–5 V therefore spans about C1–C6.
- The Rate / Fine knob (4) becomes a bipolar ± 1 -octave fine tune (12 o’clock = centred), so you can nudge the base pitch up to an octave either way — extending the reach and letting you fine-tune to another oscillator.
- Tuning is codec-corrected so notes land true (the same correction validated by the AA 440 Hz reference).
- For pitched playing without a V/oct source, you can still drive the Sync input (10) from a VCO (audio-rate hard sync); there the Rate knob shapes timbre.

A440 reference (AA slot)

Past F99 sits a special AA slot — a built-in A = 440.0 Hz tuning reference. Select it on Out 1 (1-AA) and Ogham emits a steady 440 Hz tone for tuning oscillators or your ear.

- The pitch is fixed and exact — it ignores the Rate / Fine knob, external clock, and V/oct, so you can’t accidentally detune it.
- It’s tuned in firmware against the real hardware, reading 440.0 Hz on a meter.
- Use it on Out 1 (the primary voice, which clocks the engine); on Out 2 it plays but is not guaranteed to be 440 Hz.

Outputs (12–15)

Output	Carries
Out 1 (12)	Voice 1 — the primary voice. Shaped by the Tone macro + FX.
Out 2 (13)	Voice 2 — an independent formula, same Tone + FX (unless frozen via Drone).

Output	Carries
ENV Out (14)	Mode-selectable (Menu) — envelope follower of either voice, or a raw bytebeat as a DC/LFO source. See below.
EOC (15)	End-of-cycle pulse (≈ 10 ms), BPM-synced to Voice 1's detected tempo.

ENV Out modes (set by the **C.** field in the Menu). Unlike the AC-coupled audio outputs, this jack is DC-coupled (Daisy DAC → gain stage → jack), so it can hold slow and static voltages:

Mode	Output
0.En1	Envelope follower of Out 1 — tracks its amplitude/loudness contour (fast attack, slower release, extra smoothing). Patch to a VCA/filter for amplitude-linked modulation.
0.En2	Envelope follower of Out 2.
0.dc1	Raw Out 1 bytebeat as a DC voltage (pre Tone/FX). With a slow formula/Rate this becomes a bytebeat LFO with strange, complex shapes — something the audio outs can't do (they're AC-coupled and droop below ~ 1 – 2 Hz).
0.dc2	Raw Out 2 bytebeat as a DC voltage. Pair with Drone for a frozen, self-running modulation source.

Voice 1 is “primary”: the EOC (and the default ENV envelope) follow Out 1, so patch the voice you want to drive modulation/clock from there.

Display reference (7)

Shows	Meaning
1-NN / 2-NN	Selected output (1/2) and its function number (0-based, 00 – 99)
1-AA / 2-AA	Selected output playing the A440 tuning reference
F.on / F.off	Menu: global FX enable / bypass
CL.45 etc.	Menu: FX stage field T x. + value
F.Ser / F.Par	Menu: chain topology (series / parallel)
q.oFF ... q.128	Menu: A/B smooth (interpolation grid)
d.on / d.oFF	Menu: Out 2 drone / freeze
0.En1 ... 0.dc2	Menu: ENV Out mode
Sr.NN / SF.NN	Menu: ENV Out slew, rise / fall
H.oFF ... H.256	Menu: ENV Out hold (DC modes)

Shows	Meaning
<code>Lp.oF</code> / <code>Lp.NN</code>	Menu: LPG off, or on with that decay
<code>t.oFF</code> / <code>t.A</code> / <code>t.b</code>	Menu: CV→Tone routing
<code>A nnn</code> / <code>b nnn</code>	Brief flash of Param A / B as you adjust it (~1 s)
far-right dot	Tone knob is at the clean center

Power-on sequence

At power-up Ogham shows, in order:

1. A startup animation — a single segment chases the outer border of the display for about a second.
2. The firmware version — `M.mm` for about a second: `1.03` is major version 1, minor 03. Worth noting before you update, since it identifies the installed build. The module also stores it, so a later update can tell which version it is replacing.
3. Normal operation — the function display, and audio begins.

Firmware 1.00 predates the version display and shows nothing at step 2; if the animation runs straight into the function display, that is what you have. See [Section 12](#) to update.

To factory-reset, hold the Func encoder through this sequence (see below).

Persistence & factory reset

- Your two function selections and the entire Menu (FX levels/types/params, topology, global on/off, smooth `q`, drone, ENV-out mode, Tone-CV routing, LPG on/off and decay) are saved to flash and restored on power-up (a few seconds after you stop changing them). A frozen drone's pitch and A/B are saved too.
- **Factory reset:** hold the Func encoder while powering on → both voices return to `F0` and the Menu returns to defaults.
- Knob-driven values (A, B, Rate / Fine, Tone) always follow the live knobs and are not stored — the panel always reflects what's playing.

Parameter range summary

Control	Range / values
Function (per voice)	<code>F0</code> – <code>F99</code> (0-based) + <code>AA</code> reference; no wrap, accelerating
Param A / B (knob + CV)	0–255, shared by both voices
CV A / CV B	±5 V bipolar, summed with the knob (or routed to Tone)

Control	Range / values
FX chain	3 stages (Chorus/Flanger/Phaser); per stage: level 0–99, type 0/1, 2 params 0–99; global on/off; series/parallel
Smooth q	off / 4 / 8 / 16 / 32 / 64 / 128 (A/B interpolation grid)
Drone	Out 2 coupled / decoupled (freeze)
ENV Out mode	0.En1 / 0.En2 (envelope) · 0.dc1 / 0.dc2 (raw bytebeat DC)
ENV Out slew	rise and fall independent, off – ~5 s each (all modes; DC modes add automatic level compensation)
ENV Out hold	off, or sample-and-hold every 2–256 steps (DC modes)
Tone CV	off / CV A → Tone / CV B → Tone
LPG	one field: off, or 1–99 ≈ 2 ms – 20 s decay (knee at 50 = 10 s), triggered by Sync
Rate / Fine	1/64×–64× (1× at noon); quantized clock ×/÷ (to ±32) when clocked; ±1-octave fine tune in VOct mode
Tone	Bipolar; center = clean
Mode switch	Clk (tempo) / VOct (pitch)
Clk input	Tempo/clock: clock rate = 1× native; Rate knob = quantized ×/÷ (to ±32); holds last tempo on unplug
VOct input	1 V/oct, 0–5 V (0 V = C1), ~5-octave hard-synced pitch
Sync	Rising-edge hard-sync reset (also plucks the LPG when it is on)
ENV Out	0–5 V envelope, or DC bytebeat (full range)
EOC	~10 ms end-of-cycle / BPM-synced pulse

6. Working with two voices

Ogham is really two bytebeat voices sharing one time base:

- Out 1 (12) = $\text{formula1}(t)$, Out 2 (13) = $\text{formula2}(t)$.
- By default they share the Rate, A/B params, the Tone macro, and the FX chain, but each plays its own function on its own output.
- Each voice keeps independent effect state, so the FX (chorus/flanger/phaser) give Out 1 and Out 2 their own movement — a natural stereo spread.

To set them up: turn Func to set Out 1's function → short-click to Out 2 → turn to set its function → short-click back to Out 1 as needed.

Decoupling Out 2 (Drone). The Menu's `d.on` toggle breaks the link: Out 2 freezes into an independent drone at its current pitch and A/B, bypassing Tone and FX, while Out 1 stays fully live. This turns the pair into a held drone + a performable lead (see Section 5, *Drone*).

Ideas this enables

- Two different formulas → a layered/stereo pair from one module.
 - Same formula on both → a doubled, FX-widened mono-to-stereo voice.
 - Freeze + perform → `d.on` a lush Out 2 drone, then solo over it on Out 1.
 - CV/gate from Out 1 only → use Voice 1 as a "lead" that also drives the envelope and EOC outs, with Voice 2 as a free companion texture.
-

7. Patch ideas & use cases

Voice / melodic

- **Layered lead + texture:** different formulas on Out 1 and Out 2 to separate channels for an instant stereo/dual-voice patch; sweep A/B to morph both.
- **V/oct melodies:** switch Mode → VOct, patch a sequencer/keyboard's 1 V/oct into the Clk/VOct jack, and play Ogham chromatically (0 V = C1); use the Rate / Fine knob to fine-tune to the rest of your patch.
- **Plucked notes:** switch the LPG on (**Lp. 20**), i.e. on with a short decay) and drive Sync from a trigger sequencer — each trigger restarts the waveform *and* plucks the gate, so Ogham plays discrete percussive notes instead of a continuous stream. Add V/oct on the Clk/VOct jack and you have a playable tuned instrument.
- **Drone + pluck from one module:** LPG on, drone on (**d.on**). Out 2 holds a frozen drone (ungated) while Out 1 plays plucked notes over it.
- **Hard-sync lead:** patch a VCO (saw/pulse) into the Sync input and take Out 1 — Ogham becomes a hard-synced timbre that tracks the VCO's pitch; A/B shape the sync'd timbre.
- **Tune up:** select **AA** on Out 1 for a 440 Hz reference to tune your oscillators.

Rhythmic / clocked

- **Tempo-locked patterns:** Mode → Clk, clock the Clk input from your master clock (a few Hz) so Ogham's rhythm rides your tempo; use the Rate / Fine knob to pick a clean multiply/divide ($\div 32 \dots \times 32$) for half/double-time feels, and reset phase per bar with the Sync input. Pull the clock and it holds the tempo.
- **Rack clock source:** take the EOC out (BPM-synced to the playing formula) to drive sequencers, dividers, or envelopes — a generative clock from the music itself.
- **Drum/percussion voice:** short, sync-reset hits — trigger the Sync input from a sequencer so each step restarts the waveform; pick percussive formulas and use Tone CW for crunch.

Timbre / texture

- **Modulation FX:** long-press into the Menu and bring up Chorus for width, Flanger for sweeps, or Phaser for swirl — try the Barber-pole flanger for an endless rising sweep, or the Ensemble chorus for a lush detuned spread. Flip **F. Ser** / **F. Par** to hear series vs parallel.
- **Hands-free Tone (Tone CV):** set **t.A**, patch an LFO/envelope into CV A, and let it sweep the whole clean→crush macro while knob A sets Param A.
- **Tone performance:** ride the Tone knob live — CCW for warm folded leads, CW for thin/clicky/decimated breakdowns, snap back to center for clean.
- **Evolving drone:** set a low Rate, hold a formula, then **d.on** to freeze Out 2 as a self-running drone while you keep playing Out 1.

Modulation / system integration

- **Bytebeat LFO out:** set the ENV Out to `0.dc1` (or `0.dc2`), drop the Rate low, and use the resulting slow, strange bytebeat waveform as a DC modulation/LFO source for filters, VCAs, or wavefolders elsewhere — the audio outs can't hold these slow shapes, but this DC-coupled jack can. Freeze Out 2 (`d.on`) + `0.dc2` for a stable, independent LFO. Add Hold for a stepped sample-and-hold voltage and Slew to glide between the steps; nudge the Rate knob to re-roll the shape if a setting feels dull.
- **Envelope follower:** set `0.En1` (or `0.En2`) and take ENV Out into a VCA, filter cutoff, or VCO index — so Ogham's dynamics shape another voice (auto-wah, ducking, amplitude-linked timbre).
- **Animated timbres:** patch LFOs/envelopes into CV A and CV B (centered knobs) for bipolar modulation of the formula parameters.
- **Generative duo:** cross-patch — Ogham's EOC clocks a sequencer whose CV drives Ogham's A/B, and the Sync input is reset from a slow divider — a self-evolving system that never quite repeats.

Quick-start patch

1. Out 1 → mixer/speakers. Tone at 12 o'clock (clean).
 2. Turn Func to choose Out 1's function; sweep A/B to taste.
 3. Rate / Fine knob to set speed (12 o'clock = native).
 4. Short-click Func → set Out 2 to a second function; patch Out 2 to another channel for stereo.
 5. Long-press Func into the Menu; raise a stage's level and tweak its params (try Chorus), then long-press out.
 6. Patch a clock to the Clk/VOct jack (Mode = Clk) or a 1 V/oct source (Mode = VOct), and a trigger to the Sync input (reset/sync).
-

8. Clocking, pitch & sync in depth

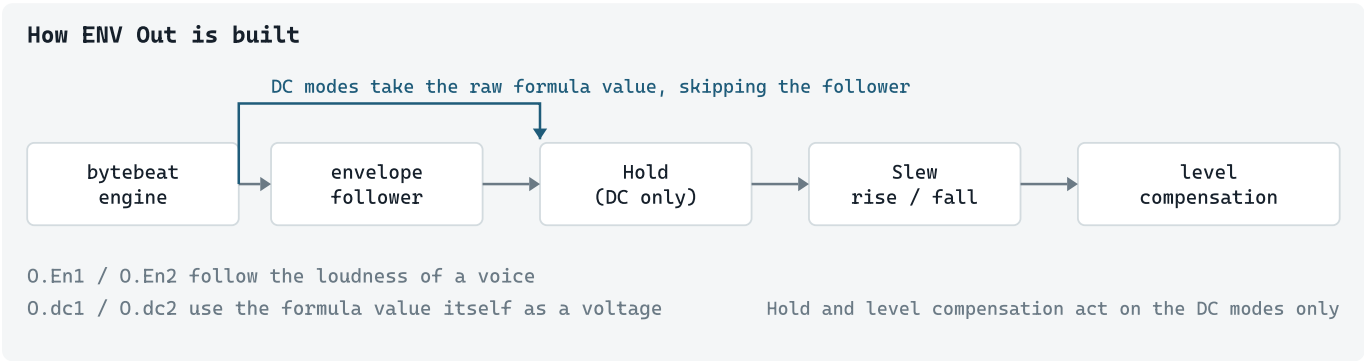
Ogham gives you three ways to control time and pitch, and they can be combined.

- **Free-running:** with nothing patched (Mode = Clk, no cable), the Rate / Fine knob sets speed ($1/64\times$ – $64\times$, $1\times$ at noon).
- **Tempo / clock (Mode = Clk):** the external clock sets the playback rate (the clock rate plays at $1\times$ native; faster = faster). The Rate / Fine knob is a quantized multiply/divide of the clock — $\div 32 \dots \times 1$ at noon ... $\times 32$ in powers of two. On unplug Ogham *holds* the last tempo and keeps playing (a later edge re-locks; nudge the Rate knob to return to continuous control).
- **Pitch / V-oct (Mode = VOct):** a 1 V/oct CV plays clean tuned notes via internal hard-sync, tracking ~ 5 octaves with $0\text{ V} = \text{C1}$ ($0\text{--}5\text{ V} \approx \text{C1--C6}$). The Rate / Fine knob becomes a ± 1 -octave fine tune. Tuning is codec-corrected (validated by the AA 440 Hz reference).
- **Hard sync (Sync input):** a rising edge restarts the waveform sample-accurately. At audio rates this is true oscillator hard sync — the waveform repeats at the input pitch for a clean, tonal note (here the Rate knob shapes *timbre*). At musical rates it's a phase/one-shot reset per trigger. A great pitched path if you don't have a dedicated V/oct source.

The Clk input sets the *tempo*; VOct sets *pitch*; the Sync input *restarts the waveform*. Clk and VOct share one jack (selected by the Mode switch); Sync is separate and can be combined with either (e.g. V/oct pitch + a per-bar Sync reset).

9. Deriving control voltages

ENV Out can do two quite different jobs, and the difference matters more than the name suggests. It can follow a voice’s loudness, which behaves like any envelope follower you’ve used before. Or it can use the bytebeat’s own value as a voltage, which behaves like nothing else — and comes with a few consequences worth understanding before you decide something is broken.



The two families

Mode	What comes out	Good for
O.En1 / O.En2	The amplitude contour of Out 1 / Out 2 — rectified, smoothed, always positive.	Ducking, auto-wah, opening a filter in time with the part. Predictable.
O.dc1 / O.dc2	The formula’s own value, straight out as a voltage (before Tone and FX).	A modulation source with a character you won’t get from an LFO — but see the gotchas.

The envelope modes are the safe choice: whatever the formula is doing, its loudness contour is smooth and well-behaved. The DC modes are where the interesting material is, and where the rest of this section applies.

Hold and Slew

Two stages shape the DC modes, in this order.

Hold (

H.off

 ...

H.256

) decides how often the voltage updates. Off, it follows every step of the formula. Set to a window, it takes one value per window and sits on it — a sample-and-hold clocked by the bytebeat itself. The steps stay in time with the pattern.

Slew (

Sr.NN

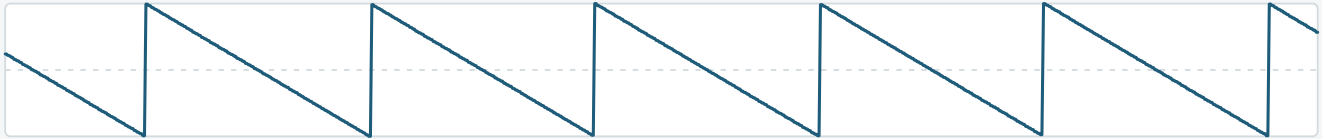
 /

SF.NN

) smooths the movement, rise and fall independently. It’s a plain time — roughly 3 ms to 5 seconds — and means the same thing at any Rate, so a setting you like stays put as you speed the engine up or down.

Hold and Slew

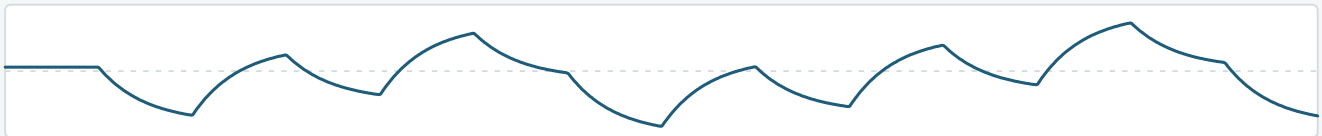
Hold off — follows every step



Hold 32 — one value per 32 steps



Hold 32 + Slew — glides between the held values



the same 448 formula steps in each row

Together they cover most of what you'd want: raw for something jagged and detailed, Hold for a stepped sequence locked to the music, Hold plus Slew for a smooth wandering voltage.

Slew also compensates its own level in the DC modes. Smoothing a signal shrinks it toward its average — that's unavoidable — so Ogham measures how much it lost and puts it back. There's a ceiling: smooth a very fast LFO hard enough and there is genuinely nothing left to restore, and the output goes quiet. That's physics, not a fault.

Gotcha 1 — where the sample lands decides everything

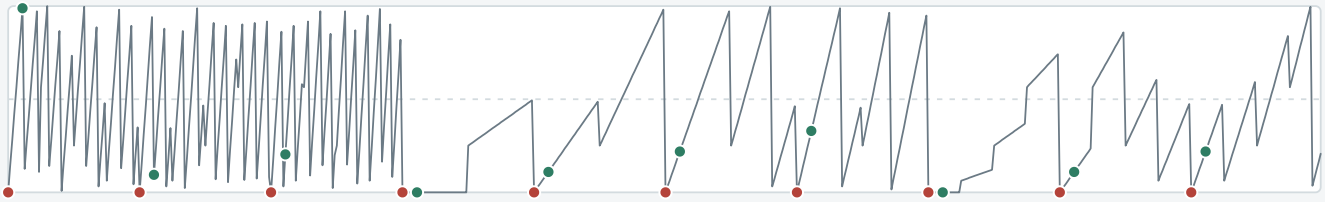
This is the one that surprises people, and it's a property of bytebeat itself rather than of Ogham.

Most formulas multiply by τ and keep the low 8 bits. The hold window is a power of two. So if the sample were taken exactly on the window boundary, those low bits would be zero *every single time* — and the output would be a constant. Not a dull voltage: a flat line.

Why the sample point matters

One function, hold window 64. Same function, same window — only the sample point differs.
the formula itself, every step

● sampled on the boundary ● sampled 7 steps later



Result — every sample lands on zero, so the voltage never moves



Result — the same formula, sampled 7 steps off the boundary



Ogham never samples on the boundary — this is what it is avoiding

The top row is the formula itself, every step. The red dots mark where a boundary sample lands — every one of them on zero, which is the whole problem. The green dots are the same formula and the same window, sampled seven steps later, and they land all over the waveform. The two rows below are the voltages those two choices produce.

Ogham always samples off the boundary, so you should never see the flat case. It also picks a new sample point each time you move the Rate knob, because which point you land on changes the character of the result quite a lot and there's no way to know a good one in advance. If a setting feels dull, nudge Rate and you get a different draw of the same formula at the same speed. That is the intended way to hunt for something you like.

Gotcha 2 — some functions can only make a handful of voltages

A bytebeat formula was written to sound interesting, not to make a good control voltage, and those are different jobs. A function can sound rich and still be built from very few numbers — and if there are only a few numbers, there are only a few voltages.

The cause is usually a single operator, and it shows up plainly if you plot a function on top of every voltage it can reach:

How many voltages a function can make

Faint lines mark every value a function can reach — the trace can only ever sit on them.

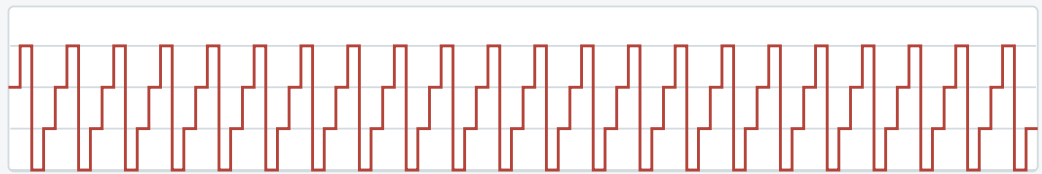
4

voltages

multiples of 64 — a mask leaves the multiplier just four values

$$t \times ((t \gg 11 \times A) \& B) - 126$$

$$A = 176 \quad B = 204$$



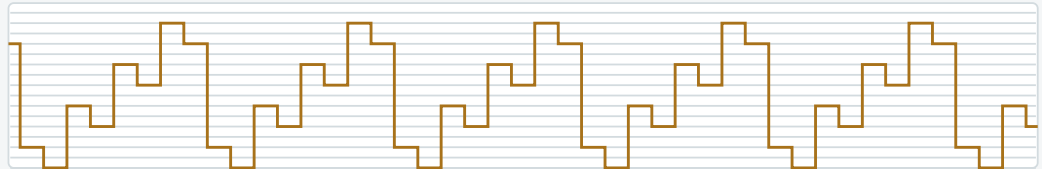
16

voltages

multiples of 16 — a final AND with one side shifted left by four

$$((t \mid t \gg 1) \div A) \& (((B \% t) ^ (23 \% 2t) + t) \ll 4)$$

$$A = 101 \quad B = 99$$



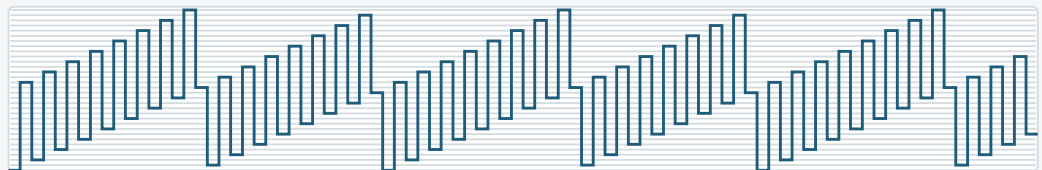
32

voltages

multiples of 8 — the same formula, opened up by a different A and B

$$t \times ((t \gg 11 \times A) \& B) - 126$$

$$A = 120 \quad B = 190$$



The faint lines are the complete set — every value the function is capable of producing, not just the ones in view. The trace can only ever sit on them.

Each panel prints the function's actual expression — taken from the current bank — along with the A and B it was drawn at.

The first manages just four voltages. The mask in `(t >> 11 × A) & B` leaves its multiplier only four possible values, so the output jumps between four rails with nothing in between: closer to a four-step sequencer than an LFO. The second reaches 16, where the `<< 4` at the end of its expression forces every value onto a multiple of 16.

The third is the same expression as the first, with different A and B — and it reaches 32. That is the clearest demonstration that the knobs matter as much as the choice of function: one setting gives a four-step stepper, another gives something eight times finer, from identical arithmetic. If a function looks promising and still won't move, try A and B before giving up on it.

Even 32 is a staircase rather than a contour once spread across the output range, and Slew will not fill the gaps — it smooths the path between values, it cannot invent values the function never produces. The better functions in the bank reach all 256 and sweep smoothly; these three cannot, whatever you do downstream.

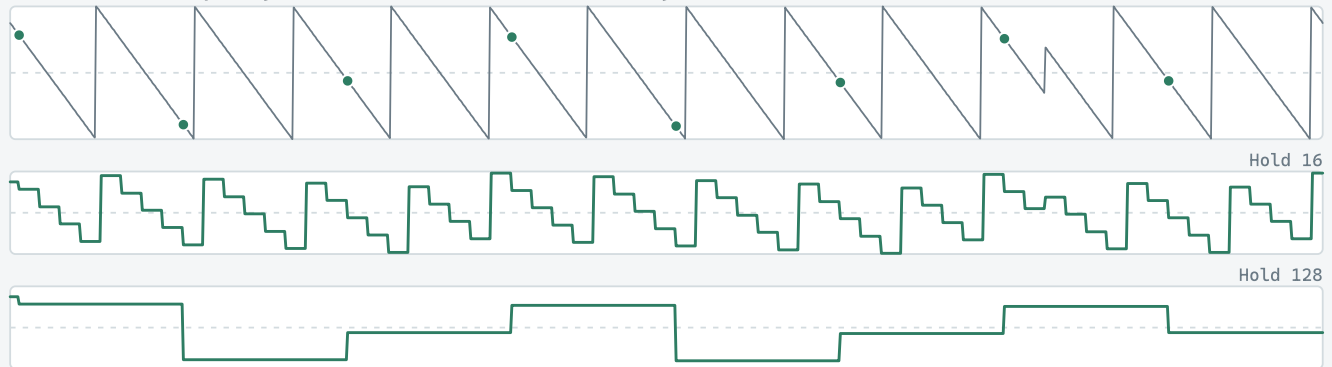
Gotcha 3 — a longer Hold changes the timing, not the range

Widening the hold window makes the voltage step less often. What it does *not* do is give the function more values to reach — that is fixed by the function and its A/B setting.

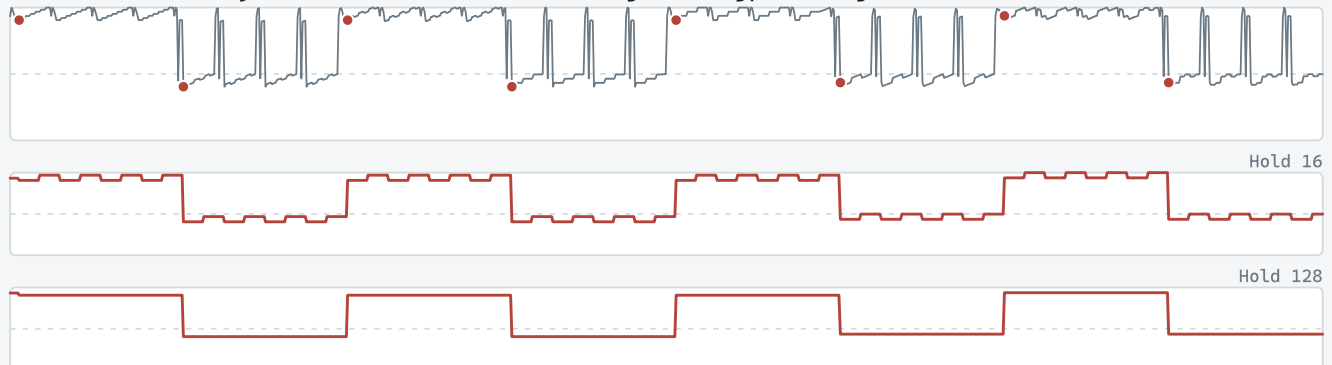
What a bigger Hold window does

Hold 16 against Hold 128, on the same stretch of each function. Dots mark the Hold 128 samples.

A function with plenty of levels – a shorter window buys detail



A function with only a few levels – the window changes timing, not range



Same range in both rows of the lower pair – only the step rate changed

The top pair is a function with plenty of levels. A short window traces a detailed contour; a long one picks a few points out of the same shape.

The bottom pair is a function with only a handful. Both windows cover about the same range, and all the longer window does is hold each value for longer.

That second case is the usual reason for “*I changed Hold and nothing happened*”. If the function only has a few voltages in it, every window setting looks much the same — the steps simply get wider. The fix is a different function, or different A and B, not a different window.

A long window can also actively lose range on some functions, because it keeps landing on a similar part of the pattern. One in the bank reaches 64 distinct values at a 16-step window and only 4 at 256; another drops to 2. If a long Hold goes lifeless, shorten it or nudge Rate to re-roll the sample point.

Getting a good result

- Find a function with range before tuning anything else. Put the CV on a scope or a slow filter sweep and step through functions with Hold off — the usable ones move over most of the range in many small steps, rather than flicking between a few levels. It is worth a minute; it decides everything that follows.
- Hold sets the rhythm, Rate sets the speed. Drop Rate well below noon for LFO territory; the free-running range goes down to 1/64×.

- Add Slew last, once the stepping is doing what you want.
 - If it looks dull, nudge Rate before changing anything else. It costs nothing and re-rolls the sample point.
 - If you want something guaranteed well-behaved, use an envelope mode instead. `0.En1` will always give you a usable contour.
-

10. Troubleshooting

No sound at all, but the module boots and the Menu works

Before suspecting a hardware fault, check the internal LPG (Menu field 18, `Lp.NN` / `Lp.oF`). With it switched on and nothing patched to Sync, the gate sits fully shut on both outputs — that is the LPG working as designed, not a fault. It plucks once when you switch it on so you can hear the setting, then goes quiet again until the next trigger.

- Long-press Func into the Menu, navigate to field 18, and switch it off — sound returns immediately, since the LPG is the last stage in the chain.
- Or patch a trigger/gate into Sync to pluck it.

If it is back on after a power cycle, that is the saved setting doing its job, not a bug — see [Persistence & factory reset](#) above. A factory reset (hold Func at power-on) clears it back to off.

See [The LPG](#) above for the full behaviour.

11. Specifications

- **Platform:** Electro-Smith Daisy Seed (STM32H7), custom Eurorack PCB.
 - **Format:** 10 HP (50.5 × 128.5 mm panel), 32 mm deep behind the panel including power-cable clearance — skiff-friendly.
 - **Voices:** 2 independent bytebeat voices, shared time base; Out 2 decouple/freeze.
 - **Functions:** 100 slots (**F0** – **F99**), all filled (8 kHz base, interpolated to 48 kHz output) — twenty each of textural, noise, percussive, rhythmic and melodic — plus an **AA** A=440 Hz tuning reference. See Appendix A for the bank as shipped.
 - **FX:** 3-stage modulation chain (Chorus / Flanger / Phaser), each with a clean and a characterful variant; per-stage mix, series/parallel, global bypass; A/B interpolation (“smooth”).
 - **LPG:** internal low-pass gate (coupled VCA + 2-pole filter), sharp attack, decay 2 ms – 20 s, triggered from the Sync input.
 - **Audio:** 2 outputs (Out 1 / Out 2), ~10 Vpp (±5 V, Eurorack level), AC-coupled.
 - **CV in:** CV A, CV B (±5 V bipolar, summed with knobs, or routable to the Tone macro).
 - **Pitch / clock in:** shared Clk/VOct jack (Mode switch) — external tempo clock, or 1 V/oct (0 V = C1, ~5 octaves).
 - **ENV out:** DC-coupled; selectable envelope follower (of Out 1 or Out 2) or raw bytebeat DC/LFO (of Out 1 or Out 2).
 - **Sync in:** hard-sync reset (edge-triggered, up to audio rate).
 - **EOC out:** end-of-cycle / BPM-synced pulse (~10 ms).
 - **Display:** 4-digit 7-segment (TM1637).
 - **Power:** Eurorack ±12 V (with an on-board +5 V regulator for the Daisy); +3.3 V from the Seed.
 - **Controls:** rotary encoder (Func) + 4 pots + Mode switch; settings persist to flash.
-

12. Updating the firmware

Ogham's firmware can be updated over the same USB socket the Daisy Seed uses for power — either from your browser, with nothing to install, or with `dfu-util` if you would rather drive it yourself. Both write the same image by the same route; the browser simply does the talking for you.

Check what you are running first: the version appears on the display at power-up (see *Power-on sequence*, Section 5).

Putting the module in bootloader mode

Both methods need the Seed in its DFU bootloader:

1. Power down the case and remove the module.
2. Connect the Daisy Seed to your computer with a USB data cable — a charge-only cable will not enumerate.
3. Hold BOOT, tap RESET, then release BOOT.

Nothing visible happens: the display stays blank and the module makes no sound. That is the bootloader, not a fault.

This bootloader is in the STM32's factory ROM and cannot be overwritten. An interrupted flash leaves a module that will not start, but never a dead one — BOOT + RESET always brings it back here to try again.

Flashing from the browser

Open the [firmware page](#) and use the flasher there. It needs Chrome, Edge or Opera on desktop: Firefox and Safari have not implemented WebUSB, and the page will tell you so rather than fail silently. It also only works over `https://` — WebUSB is unavailable on an insecure origin, and the page distinguishes that case from an unsupported browser, because the fix is a different one.

Click Connect module and the browser shows its device chooser. The list is filtered to devices identifying as `0483:DF11` — the STM32 bootloader — so a module in DFU mode is normally the only entry, and there is nothing to identify by guesswork. Pick `STM32 BOOTLOADER`, then Flash.

The page erases only the sectors the image occupies, writes it, and verifies the size and checksum of what it sent. When it finishes, power-cycle the module.

On Windows the module may not appear in the chooser at all. Browsers can only reach USB devices through WinUSB, and Windows usually binds ST's own DFU driver first. [Zadig](https://zadig.akeo.ie/) (<https://zadig.akeo.ie/>) replaces the driver for `STM32 BOOTLOADER` with WinUSB; reconnect afterwards and it appears. macOS and Linux need no driver setup.

Flashing with dfu-util

Download the `.bin` from the firmware page, then:

```
dfu-util -a 0 -s 0x08000000:leave -D ogham-v1.03-encrev.bin
```

The `:leave` matters. Without it the module stays parked in DFU mode after writing — blank display, apparently dead — and, because USB keeps it powered, a rack power cycle will not shift it. A trailing `Error during download get_status` is the device detaching as it restarts, not a failed write.

`dfu-util` does not verify by default. To check the image landed intact, read the flash back and compare it against what you downloaded:

```
dfu-util -a 0 -U readback.bin -s 0x08000000:111144  
cmp readback.bin ogham-v1.03-encrev.bin
```

Each release on the firmware page publishes its SHA-256 and exact byte size, so you can confirm the download before flashing as well as after.

After flashing: unplug USB before power-cycling

The Seed is powered by USB and by the Eurorack bus. With the USB cable connected, powering the case off and on does not actually reset it — the Seed never loses power, so the new firmware never starts, and the module can look broken when it is fine.

Unplug USB, then power up from the rack. You should see the startup animation, the version, and normal operation.

If the encoder turns the wrong way

Some modules use an encoder with its A/B phases swapped, and there are two builds of each release to match. If the Func encoder counts down when you turn it clockwise, you have flashed the other one — download the alternative from the firmware page and flash again. Nothing else differs between them.

Appendix A — the formula bank

The hundred functions of the bank, **F0 – F99**, written as the module actually evaluates them. **t** is the sample counter (8 kHz for this bank), and **A** and **B** are the two knob values, 0–255, substituted live wherever they appear.

The bank is grouped by character: twenty of each type, in slot order.

Reading them:

- Arithmetic is 32-bit integer, with JavaScript **ToInt32** semantics, and only the low 8 bits reach the DAC — so every expression is implicitly **& 255**.
- Division truncates toward zero; division or modulo by zero yields 0 rather than faulting.
- Shift counts are masked to 0–31. That is why **t >> 139** is really **t >> 11**, and **t << 98** is **t << 2**. Several of these depend on it.
- Where a formula does not mention **A** or **B**, that knob does nothing on that slot — that is the formula, not a fault.

Textural — **F0 – F19**

Slot	Name	Expression
F0	Electric Drone	$(B \mid \sim(t + B)) / A \wedge A * B \% (A \& t + 7) * (B + t) / 16384 + t$
F1	Lofi Drone	$B / 155 + (t \wedge (t / B / (56 / ((189 \& B - t) \% t \& (A - (B \mid A \mid (B \& 300) \% 142 \wedge t) + 41 - B \& t))) \wedge t) - 188) - (\sim 2048 + t + B) \% A$
F2	Disolving Beat	$((A \wedge t) + t + (B + \sim t) + (51 \ll 11) / (t + t) \wedge -(126 + 162 \wedge t) \% t) \gg t / (12000 + 600)$
F3	Sweep to Drone	$A \wedge (-(t \mid \sim(197 \& -(4000 \& A \wedge (t \mid A) / (t \mid 99) \mid 112) - A))) \gg 2500) - t \% (t / B)$
F4	Sweep to Crunch	$(t + A \% \sim t) \% (B \mid 100) \wedge ((t * B + B / ((t \ll B) \% (B - A / 3000 - (11 / (t \% \sim((A \ll t + t) \% 82) \ll B) - (A \ll t))) - 7)) / (t / A) \mid A / A)$
F5	Noisy Alarm	$t \% B + t \mid t / A \mid (((106 \gg 12000 + A) \% (((t \wedge \sim t) - t - 79) / t \& -213 \mid 100) \wedge A \gg t * B) - t / A \wedge (\sim(B - 133) \& 181 \ll A) \% t) \ll 32768 + t) - 223$
F6	Rise of the Glitch	$\sim((t - t \ll 171) \% (t \ll t)) \% t * ((t \gg 167) * t - t) / 196 + 208 + (A + (B \mid t))$
F7	Robotic Drone	$t / ((t \wedge B) - t) - (((((102 \gg (151 \mid 200 * A * t) / t) + t \mid (t \mid 37)) \gg (t \& t) / B) + (A \ll t)) \% (B / ((t \mid 101) \wedge t))) \mid 213) / B \wedge A$

F8	Bitty Drone	$B - ((t / A + t) \% (A \% t) \& t \& t)$
F9	Texture Sequence	$((t / (134 \% A) \% t \mid t - ((B + A \wedge -t \wedge 26) \% (t \& 1 / t) \mid B)) + t) \% ((t + (A >> t * ((214 >> B >> t) * t)) \& 47 \& 70) * A \mid (t << A) - (A + (B \wedge (A \mid 16000))))$
F10	Noise to Drone	$16000 \% ((A \& t) - (A * B / t \% B \& t \mid (-t \mid (B + (t \wedge t * B))) / A - 179 \& 68 \& A) \& B) - (500 + t)) \wedge t \% 196$
F11	Falling Down	$(t >> 2500 \wedge t) \& (1024 * B / t \% t * \sim\sim t - 100 + B \mid (B << 71) \% t)$
F12	Power Drone	$(B \mid t) \% 500 \wedge t \% -A$
F13	Run Away	$(t \% (\sim(((t \& -B >> A) / B + t) / (t \% t - (t \% (141 >> B) + 122) \mid 198 >> -(t / B) \mid A)) + t * (t / t) - 4096) * (-63 + 10000 * 145 \mid -110) + \sim t \wedge t) - 128$
F14	Slowly Rising	$(53 * ((50 >> (173 * 148 \wedge t)) * t - t) \% t \& -193) + (t \& 32000 << 102) - (((t * (t / -A) / B \mid 68) \wedge 172) - (600 + t) + 208) \& (B \mid t \wedge t << 108)$
F15	Failing Electro Loop	$((t \wedge 2000) >> A) + ((A \& t) + (136 << t \% 12000 / A) - (t / B \mid 14))$
F16	Ring Descent	$t \% (\sim((196 \% ((t * t \mid t) \wedge B / A) << 27) / B / t \% ((\sim(t - t >> A) - 131 * A << 27) / B / t << (A \& (66 \mid t)))) + t * (t / t) - 4096) * ((-63 \& B * 129) / -110)$
F17	Changing Times	$(t \% t \wedge A - t \% 10000 + ((\sim t \& t) + t)) + (t \& t \% \sim B)$
F18	Scanner	$((t \wedge t) - (t + (((t \& t \mid t) - B) \% t \wedge A) \% A \wedge -(212 \& B)))) \% (t \wedge t / (t \wedge \sim(130 - (\sim t >> A))))$
F19	Texture from Tone	$(t \wedge (t * B - ((t \wedge (156 \mid (B \wedge -(70 << 86)) * 39)) \% 16384 + t) \mid t + t - A) >> A) - t \mid (B \% (-(A * 114 << A) - (t \& B + t + -A) \mid A) \mid (t + t) / A) / B$

Noise — F20 – F39

Slot	Name	Expression
F20	Crunch Pulse	$(143 \wedge ((\sim 32768 / t \& ((\sim(B \% 151) >> t) + A << (7 >> 115) + 206) - 157 \% 20) - ((A \mid t) - t) \wedge B) \wedge t) \% 208$
F21	Electronic Beating	$17 - (B \& t \mid t \% \sim((A - (t + t)) \% \sim(B * 82))) / B + (A << t)$
F22	Fluttery	$((t + t) * A / B + t / (B \mid t) \mid t / A) + B \% ((A >> (\sim t \& t \% t)) \% t >> t / 13 << t * \sim t)$

F23	Circuit Bent	$44100 \wedge t \% \sim(111 * 50 + (t * 116 \& t << (t / 6 \& A + t) / -A) - B)$
F24	Where is my Filter	$((A \wedge A) + B \wedge (B + (-(t - A) >> t - t \% \sim t))) * (A \% (t - 131) \& 102) \& t) \% 6000$
F25	Decent Into Chaos	$16000 \% (B - (t + \sim 6000)) \wedge (A - \sim t) * (t - (-(t \% (((16000 \% (162 \wedge t \% \sim t) >> 16000 * (t << 78) >> 16384) + 52) * t) << B))) / (t / ((B / 26 A) * 155))$
F26	Pulsing static	$((((t \wedge t \% (t << -(t - t))) + t) / 100 \% (t >> 7 \wedge (112 \& A) + 89) \wedge 93) - t \% (B * t \& B) 200 + t + \sim t \% A$
F27	Hunt The Wumpus	$(B >> t >> t / (2000 + B)) - (((\sim t >> A \& t) * B << (t >> (2000 B)) >> t / (2000 + B)) - B)$
F28	Harmonic Noise Seq	$B + ((176 * (\sim B + (175 / t - 167) A \wedge 97 + t) >> t / (2000 + 190)) + ((A * t >> t / (179 \wedge (-B + A A \wedge 2048 \% t))) - 16000))$
F29	Gritty Drone	$400 + (t t + \sim(B * (A \% t \% \sim(t << A))) \& 4000) - (t \% (100 \& A) (t (t + A - (101 t * (\sim 170 \& 30) * 33) t - B)))$
F30	Red Noise	$(t >> B) * (t << t / -A - t B + t - (t t)) t - t \% (A + (t >> (196 \wedge t >> A) << t))$
F31	Evolution of Noise	$(A - t - (89 << t / (B + 189) \& (45 \& t) / t) \wedge t) \% B t >> 1000$
F32	Riser Noise	$((t * (t / -A) / B 68) \wedge 120) - (B \wedge t)$
F33	Fluttering Grind	$(B + (t \wedge t * B)) / A$
F34	Frying Pan	$B + ((B + (t \wedge t * B)) / A - 179) \wedge t \% B$
F35	Destructive Riser	$(t * (t / -A) / B 68) + (t - A)$
F36	Oblivion	$(24000 \wedge \sim(t / 40 - (104 + (t \& t)))) - 5000 / (t / A - (89 \wedge t) \% B)$
F37	Jet Phase	$8000 / (8000 / (t / A * ((B \wedge t) \% B)))$
F38	Broken Morse	$-((98 + A * -195) * (t \% (-(A \% (\sim t - A + t) \& t) A) \& t \% (600 \wedge B))) / t + (B ((t B) \& 44100) \% 30)$
F39	Grunge Gate	$116 \wedge \sim(B \wedge (t \% A + 24000) \% t) * (((-(3000 \% t + 102) \% B + A) \wedge ((56 - B) / (A \& t) \& (t 44100) 2 - A)) \% ((t - 32000) / t))$

Percussive — F40 – F59

Slot	Name	Expression
------	------	------------

F40	Recursive Descent	$(A \wedge 141 * A) \% (A * B \% (t \wedge (t >> 70) + 32 + (186 \wedge (\sim(32 \wedge 2048) \& (t \% (t / 121) + (t \wedge B) \wedge t) >> t \mid A << -131 \% (194 / 173)))) - t)$
F41	V8 Fall	$A * B \% (-((114 - (t + t)) \% ((137 >> t + (125 >> A \mid B \wedge B)) / ((57 \mid -171) \wedge A) \wedge (\sim((\sim(-75 \wedge 209) << (t \mid B) << \sim(42 << (B \mid A)) >> t) \wedge A))) \wedge t)$
F42	Crash Breakdown	$\sim((t + 109 \mid (t * (16 \wedge A) \% ((A * B \mid \sim A - B) - 16384) / t * 79 \& 191) * A * -t) / ((t \wedge t) - B \mid 187) * t + (t - A / (((t >> 201 \& A) + (106 << 91) \wedge \sim B) << 108))) / \sim t$
F43	Decent to Melody	$9 - ((t * B + (56 \& (A - (t - t)) \% (t - (-48000 + t)) + t >> 130)) / (t / A) - (t \& t / 1024)) + (A >> t)$
F44	Phaser Snare	$((((66 >> \sim t \& t / 96) + B) / (t + (25 \wedge t) \% 1200 / 32768) \& t) \% (t \& 133) \& t \mid t / B) - t \% (t \% A)$
F45	Percussive Beat	$((105 >> -A) \% t \wedge B \wedge (B \mid A * B \% t \wedge t)) + (56 \mid \sim t) / 31$
F46	Seq Crash	$(t << (t \mid -2)) + 94 >> A \mid t \% A \mid t \wedge (16384 \& (t \& B) \wedge ((\sim 32768 \wedge A) \% t \& t + (B \wedge -t)) \% t - (1000 + 154))$
F47	Phase Fall	$(t >> 128) \% A * (218 * B \wedge B / A) / t + t \mid (t \& A) \% (164 - (B \mid t)) \& (37 \mid t)$
F48	Glitch Splash	$B * ((A << 199) \% (133 + B \wedge t) - 512 \wedge t) / t * B \wedge t - t$
F49	Hitting the Edge	$(t + ((B \mid A >> t * B) \% B \wedge B)) \% 185 \wedge (A << 71) \% t \wedge (-t >> 190) * B - (4096 \mid A) \& t + t$
F50	Dive Snare	$A \wedge (A << 53) / (t + (t \% (t \% B) \mid B)) \mid t / B$
F51	Energetic Hit	$B * A \% ((32000 - B / (-127 * A)) \% t \wedge (24000 - B * (-A \% 1200) + t) \% t \& (24000 - B * \sim A + t) \% t \mid t)$
F52	Noisy Drone Out	$16000 \% \sim t \wedge 8000 / (t / A - (600 + t) \% B)$
F53	Ringmod Hit	$(24000 - B * (-A \% 1200)) \% (5000 \% t \wedge B / A \& B + (24000 - B * (-A \% ((B \mid A) >> 32768 \% t) \% t)))$
F54	Broken Crash Tone	$-((98 + A * -195) * (t \% (\sim(102 \% (300 + t) \& t) \mid A) + t \% (400 \wedge B))) / t + A / ((95 \mid 70) / B \& \sim t)$
F55	Angry Automaton	$(t \wedge (B \wedge (t \wedge ((150 / -A \mid (t \mid t)) - (86 - 122) \mid ((\sim t \wedge \sim B) / -152 \& 145 \& t) / t) / (78 >> A) >> 10000 + t - t)) / A) * ((\sim 76 \& B * A) / -t) + \sim t \wedge t \wedge 128$
F56	Cornflake Snare	$(t << ((t \mid t) << (t << (t >> B)) / A) / (188 * (\sim 53 + 43)) << 141) / (t * t) + (223 \wedge A)$

F57	Crunch Into Drone	$(t \% (A - (171 \mid \sim B) - t \% (-t \% (t - t))) - 116 >> t \& (t \& 12000) - A \wedge A + \sim((t \wedge -50) \& t) \wedge -(A - (23 >> \sim B)) << 166) \% t$
F58	Classic Pew Hit	$B * (87 * A + \sim t) / t * 110$
F59	Aliasing Hit	$B + 2000 \% t * 127 \wedge (A \mid B + A \wedge B + 3000 \% t * 102 + 44100 \mid B)$

Rhythmic — F60 – F79

Slot	Name	Expression
F60	High Freq Seq	$(t >> 165) \% ((185 \mid t) \& t / (A * (\sim t + t))) \mid (((A \& t) + B \wedge 512 \mid t) << (86 * t << B) \mid A \& -((t \mid (\sim 180 * t \mid t) / (B \mid t) \% 97) + t) + 32768) >> (193 \mid t) \% \sim t$
F61	Hidden Melody	$(t - (((B \mid t) / A \wedge (136 - -t) / -t * t) + t \mid (92 \mid 91))) \% (t \wedge (A / t >> 8000 \mid 184 << t)) \wedge \sim(B / (t * 19)) \mid t / 512$
F62	Marching Ants	$(B * t / A \wedge (((t * (214 - 117) + A) \% B \& \sim t) - 122) * B / (31 + (A << t)) >> ((-A + -16384 \wedge B \wedge t) + 223) * B / t) + t + t) + t$
F63	Blip Chopper	$(B * t / A \wedge (t \wedge -A) / -t + t) + t$
F64	Radio Chatter	$((t >> (48000 \mid B) / 187 \& (B - A / 3000 / t) \% 1 - ((79 \mid B \wedge 48000) \& B * 133 / t << (22 \wedge t / B + 4096) * A)) + (600 * (28 + 162 - B) - \sim A \mid (222 \mid t) << A \wedge 39)) \% \sim A$
F65	Phasing Bassline	$t \& ((t - B) \% t / (32 \& 1000) \wedge (A \mid (B \& t) >> A >> t) + t) \wedge \sim t$
F66	IDM Beat	$\sim((B + t - (A \% (46 * (t * t) \& t - B) >> (B - 70 / 54) \% A \% t / (t * 300))) / 35) \wedge t \mid t \& 87 / (B \mid t)$
F67	Chaos Arping	$(B + t / ((38 \& A \mid (t << t) * 3 \& (t - t \& t) * (t * (t >> (-t \wedge t) << t) \& t)) >> 30 / B) \wedge t) \% t$
F68	Glitch Bass	$-(t \% \sim B - A) \wedge t - t / (-(43 * (t >> 206 >> B) - t) >> (44100 \wedge B) - t >> t) >> (t / 56 \wedge A >> t >> \sim(77 - t))$
F69	Pulsing Gnat	$(\sim t - t \& (68 \mid A - t \% 194) >> (25 >> 205) >> 218) - (t >> ((t - \sim 12) \% 16384 \mid t) * 51) >> t \wedge B$
F70	8 Bar Glitch Sync	$(t >> (16000 \% (B * (t \mid \sim 5000)) \wedge (8000 >> (A >> t) \mid B / ((A \mid t) >> 109 * 171)))) - B \% ((A \wedge t) \& t * 77)$
F71	Scratch Throb	$((2500 \wedge A >> \sim t) + t) \% (t - (((t \wedge 1024) * A \mid t / 49) - (16000 + (t \wedge t)) + B \% (60 / ((140 / (A >> 144 - t) << A \% t) + 176)) \% (B << 195) \mid ((-t \wedge 1200) + t) / B))$

F72	Grain Period	$(((((t * t >> (6 + (A >> 6))) \& 255)) >> ((t \% (600 + (B << 3))) >> 8)))$
F73	Cycling Hits	$((85 \wedge 109 >> 188) \% (t - (B \wedge A)) * A \% B \mid ((\sim t \mid \sim t / A) \wedge t) << (t \mid t >> 12000) << B) - (t + (48000 >> (t >> (t / A >> (B >> 50 \& B))))))$
F74	Emergency	$t / B \mid A * \sim t \mid A * B + t - A << 13$
F75	Crash and Beat	$(t - ((32768 \% t - ((t - (\sim t >> B)) * 33 \wedge (38 - 40 \mid (-t + (\sim t + A) \wedge t) \% t))) / (t >> (196 \wedge t >> A)) + 97) \mid t) - (10000 \% t - t + 159 \mid t \wedge t) + ((B \mid (t \mid t) \% A) \wedge t)$
F76	Four on the Glitch	$((((t + t) \% t << A) << 38) / 143 - (t + (B \wedge (t \wedge t / (\sim t / (-A - (\sim t + t - t)) \mid t))))))$
F77	Elemental Loop	$(t \wedge t * B) / A \mid t / B$
F78	Electro Chatter	$(t >> \sim 186) + (t \mid B) << ((t >> A << t) * ((10000 + t) / (179 / t - 91)) / t \% t \& (B \& t)) \wedge 300 \% ((t \& A \& (t \wedge 132)) >> t * 148 + t)$
F79	Algorunning	$181 \& (t \wedge A * (B \& t / 200) - ((t \wedge t) \& 8192) \wedge t * \sim(-76 + A)) \mid (185 * (2000 >> -204 \& t) \mid (B + t) / B)$

Melodic — F80 – F99

Slot	Name	Expression
F80	Melody Rhythm Pump	$3 + t - (t / A \wedge t) \wedge B$
F81	Melodic Riser	$\sim((A - (B << t \mid (\sim B \wedge \sim 102 * t) * t \% ((14 + A) \% (-B * (\sim t \% t) - t \mid t / 3)))) / 115)$
F82	Blip Sequence	$((B \mid t) / A \wedge ((t / B \mid A) \& 115 \& ((\sim(50 \wedge t) + 114) / t / -179 + t - ((t \% (\sim(t * t) + t) \wedge t) \& t)) \% (t \wedge -(A \% t \% B)) \mid t \wedge B \& 98)) + t$
F83	Phasing Drone	$(t + (t + t)) \% ((10 \mid t) / A \wedge t \% \sim(A - (B \mid (t * t \mid t)) \mid \sim(B * 79)) / t << -(B \% (t * t)) >> 4096) + \sim B$
F84	Melody Drone Beat	$A \% t >> (t << (B - t) / (76 \% 85)) \wedge (t - (B \wedge t) \wedge t >> t / B) >> t$
F85	Gently Evolving	$-(t \% -B - A) \& \sim t + (B / (((t + 111) / t \mid (214 \wedge 160) / (A << t) \% 85) \% t \mid 112 \& 32768 \% (19 >> A)) \wedge (t + (A \mid B) \% A) \% t) - (127 \mid 8192) >> (t << \sim B) - A$
F86	Ring Swing	$(((((t * (4 + (A >> 6))) * (t >> 9)) \& 255)) >> ((t \% (450 + (B << 3))) >> 8)))$

F87	Sub Harmony Chop	$(((t*2)\&255)>>1)+(((t*(4+(A>>5)))\&255)>>2))>>((t\&(1023+(B<<4)))>>9)$
F88	Melodic to Noise	$(((B \wedge t / A) - 132) \% 181 \mid (A \& ((t * t \wedge t) \% t \mid t \% (B \wedge -t))) + 3000) >> B / (\sim -32768 << A << t \mid A \% (B \wedge \sim t))) \wedge t) + t$
F89	Held Note Glitch	$A * 4096 * B / ((t \mid t << t) - (A << 400)) + ((t \& t) >> (t \wedge -25 \% (t \& t)) \% t)$
F90	Gentle Arpeggio	$(t / A \mid t + ((t / \sim B \wedge t) << A)) + (B \wedge t)$
F91	Sing Song	$96 + B \wedge (t \mid (t / \sim A \mid t \& \sim 166 << B)) + (-t - (t \wedge A)) \wedge (t \% -A \wedge t) >> (A << 204) / B \& A - 1200$
F92	Glitchy Song	$((-63 << 85 \& A) << 38) / 143 - (t + (B \wedge (t \wedge t / (\sim t / (-(1024 - (\sim t + t - t)) \mid t)))))$
F93	Classic Byte Arp	$t / A \mid t \wedge (1000 * B \mid t \% t \wedge 154) + (t * B \wedge t) \mid t / B$
F94	Pulsing Line	$((t \& A) - t / B >> (B << A \& (A + 62 << t) \% t \% t)) \% t \% t$
F95	Hidden Paths	$(44100 + A * \sim 195) * (t \% (6000 + 1000 - t >> \sim B) + 75 \% (20 / B))$
F96	Melody Gate	$(t / t + 114) * ((-42 \& B * A) / -t) + \sim t \wedge t \wedge 128$
F97	Simple Arp Notes	$(t \wedge B) - -t \% (A << 133 \wedge t) \% (t + B \mid A >> (16384 * 191 \mid 118 * B) \wedge (t - B \mid (62 \& (8000 \wedge B \& t \wedge 62) \mid B)))$
F98	Broken Song	$A - (t \wedge (t - 1) / -B) \wedge 2500 \% t << 118 \wedge (t \mid B)$
F99	Oldskool Tune	$(t \% B - (\sim B / 1200 - 21) \mid t / A) - B$

Ogham — make a lot from a little.